Native iOS Sudoku

Implementation Plan & Codex Phase Contracts | Version 3.0 | October 2026

Project: https://github.com/angel-avalos-c/Sudoku-iOS

1. Purpose, constraints, and definition of success

Build a native Swift/SwiftUI Sudoku game from scratch, learning iOS development and test automation. Develop the platform-independent SudokuCore Swift package on Windows; build the iOS app and run simulator tests on GitHub-hosted macOS runners. No paid services, hardware purchases, or Apple Developer subscription are assumed.

- Initial goal: a basic playable 9×9 Sudoku with a valid puzzle, number entry, immutable givens, rule validation, and a visible completion state.
- Not in initial MVP: accounts, cloud sync, leaderboard, monetization, analytics, hints, notes, multiple themes, or a server backend.
- One phase per Codex task; do not implement future phases opportunistically. All functionality should be deterministic and testable.
- The public GitHub repository is the source of truth. Do not commit secrets, personal device identifiers, signing credentials, or generated build directories.

2. Verified baseline and unresolved platform limits

Item	Current state
Windows toolchain	Swift 6.4.1, VS Code + Swift extension, Visual Studio C++ prerequisites; Windows Developer Mode enabled
Local package	SudokuCore Swift package; swift build and swift test pass; generated sample test passes
Repository	main pushed to public angel-avalos-c/Sudoku-iOS
CI	GitHub Actions macos-15 runs swift build and swift test successfully
Swift compatibility	Package.swift tools version set to 6.1 for macOS runner Swift 6.1; do not raise without CI toolchain update
iOS SDK / simulator	Not yet verified; CI Swift package success does not prove iOS app build or simulator tests work
Physical iPhone	Available, but free Windows-only deployment/signing is not established; do not claim it is solved

3. Architecture and repository contract

Keep SudokuCore completely independent of SwiftUI, UIKit, and Apple-only frameworks. SwiftUI target consumes SudokuCore. Do not introduce a Go backend or reuse the earlier Sudoku project code.

```
Sudoku-iOS/
  Package.swift
  Sources/SudokuCore/              # pure Swift rules, board, session
  Tests/SudokuCoreTests/           # Swift Testing unit tests
  iOS/SudokuApp/                   # future SwiftUI application sources
  iOS/SudokuApp.xcodeproj/         # future generated/committed Xcode project
  iOS/SudokuAppUITests/            # future XCUITest target
  .github/workflows/swift-tests.yml   # current cross-platform package CI
  .github/workflows/ios-tests.yml     # future iOS CI
  docs/                            # specifications and QA notes
```

Naming contract: SudokuCore is the package/module; SudokuApp is the native iOS app. Use zero-based row and column indices internally (0...8). Use integers 0...9 for cell values, where 0 means empty. Public APIs must document invalid-input behavior.

4. Workflow and Codex execution protocol

1. Read this document and inspect the repository before editing; identify the requested phase and its dependencies.
2. Create a feature branch named phase-N-short-description. Implement only the requested phase.
3. Preserve existing passing behavior; add unit/UI tests for each new behavior and meaningful edge cases.
4. Run locally available checks. If a check requires macOS, add or update CI and state clearly that it cannot be executed on Windows.
5. Report files changed, design choices, exact commands run, test results, known limitations, and acceptance checklist. Do not claim unrun tests passed.
6. Stop for human review; do not merge to main or begin the next phase without approval.

Reusable Codex request: “Implement Phase N of docs/iOS_Sudoku_Project_Plan_v2.docx (or the equivalent PDF) exactly as specified. Read the phase contract and project-wide rules, inspect the repository, create the named feature branch, implement code/tests/CI required by that phase, run available checks, report results, and stop before merging. Do not expand scope.”

5. Phase 1 — Windows toolchain, repository, and macOS package CI

Status: COMPLETE for Swift package development. Native iOS SDK/simulator validation remains in Phase 4.

Repeatable setup / recovery instructions

7. On Windows verify git --version, code --version, winget --version, swift --version, and swiftc --version.

8. Install Git and VS Code if absent. Follow https://www.swift.org/install/windows/ for the current Swift Windows toolchain and Visual Studio C++/Windows SDK prerequisites. Do not assume VS Code includes Visual Studio build tools.
9. Enable Windows Developer Mode if SwiftPM cannot create .build symbolic links. Restart shells after installations.
10. Install the official Swift extension in VS Code. In the repository root, run swift build and swift test.
11. Initialize git main if starting fresh; create .gitignore excluding .build/, DerivedData/, user-specific Xcode state, and local credentials. Commit the package and tests.
12. Create a public GitHub repository without auto-generated README/gitignore, add origin, and push main.
13. Add .github/workflows/swift-tests.yml: checkout; swift --version; swift build; swift test; trigger on push to main, pull requests to main, and workflow_dispatch; use macos-15.
14. Keep Package.swift at // swift-tools-version: 6.1 while the macOS runner supplies Swift 6.1, unless explicitly upgrading and validating the toolchain.
15. Verify the Actions run passes. Swift Testing reports the example test separately from XCTest's "Executed 0 tests"; that is normal.

```
git --version
code --version
swift --version
swiftc --version
swift build
swift test
git status
git remote -v
```

Reference workflow (existing working behavior):

```
name: Swift Build and Tests
on:
 push:
  branches: [main]
 pull_request:
  branches: [main]
 workflow_dispatch:
jobs:
 swift-tests:
  runs-on: macos-15
  steps:
   - uses: actions/checkout@v4
   - run: swift --version
   - run: swift build
   - run: swift test
```

Phase 1 acceptance: local swift build/test pass; git push works; macOS Swift package CI passes. Current baseline meets these criteria.

6. Phase 2 — SudokuCore board model and rules

Implementation contract

Deliverable: Pure Swift board representation and correct move validation.

Primary files/modules: Sources/SudokuCore/Board.swift, Cell.swift, SudokuRules.swift; Tests/SudokuCoreTests/BoardTests.swift and RulesTests.swift.

Required verification: swift build; swift test on Windows and macOS CI; invalid board dimensions/values, duplicate row/column/box, fixed-cell editing, clearing editable cells.

Acceptance gate: No UI or puzzle generation; all board operations deterministic, validated, and covered by tests.

- Represent 9×9 grid (81 cells) with value 0...9 and given/editable flag; prevent callers from mutating a given. Validate input dimensions and value ranges on initialization.
- Expose readCell(row:column:), setValue(_:row:column:), clearCell(row:column:), and conflict detection. Prefer typed errors or explicit results over silent failures.
- A move is invalid when a nonzero value duplicates a value in the same row, column, or 3×3 box. Ignore the target cell when checking its peers.
- Allow clearing an editable cell; do not silently modify other cells. Include isComplete and isSolved checks, where solved requires every cell filled and all Sudoku constraints satisfied.
- Provide fixtures: valid partially filled puzzle, valid solution, conflicting board, empty board. Document zero-based coordinates and value 0 convention.

7. Phase 3 — Puzzle fixtures and game session (MVP engine)

Implementation contract

Deliverable: Start a reproducible game from a known valid puzzle; track selection and edits; detect win.

Primary files/modules: Sources/SudokuCore/Puzzle.swift, GameSession.swift, PuzzleCatalog.swift; session/fixture tests.

Required verification: Unit tests for starting state, givens preservation, invalid entries, undo-free edits, completion, and reset.

Acceptance gate: A complete Sudoku round is playable through the core API without a UI.

- Ship at least one hard-coded, verified puzzle/solution pair; do not implement random puzzle generation yet.
- Session owns current board, original puzzle, selected cell (optional), game status (inProgress/completed), and mistake feedback.
- Define invalid move behavior explicitly: reject conflicting values and return an error; the UI will display an error without changing the board.
- Reset restores the original puzzle. Completion is emitted only after a fully valid solution.
- Do not add timer, hints, notes, saved games, or difficulty selection in this phase.

8. Phase 4 — Native iOS app scaffold and simulator CI

Implementation contract

Deliverable: A minimal launchable SwiftUI iOS app that links SudokuCore and builds in GitHub Actions.

Primary files/modules: iOS/SudokuApp sources, Xcode project, iOS/SudokuAppTests (optional smoke), .github/workflows/ios-tests.yml.

Required verification: xcodebuild -version; xcodebuild -list -project; build for iphonesimulator; simulator availability check; one launch smoke test if feasible.

Acceptance gate: Committed reproducible Xcode project and a green iOS simulator build in CI, or a documented blocker with evidence.

- Use SwiftUI App lifecycle, a single root screen, and SudokuCore as a local Swift package dependency. Target an iOS version supported by the selected macOS runner/Xcode; document the chosen deployment target.
- Because Xcode is unavailable on Windows, generate the Xcode project using a reproducible, reviewed macOS CI process (for example a pinned project-generation tool/config), or commit a project created with legitimate macOS access. Do not pretend VS Code can build an Xcode project locally.
- Add an iOS workflow with a pinned macOS image and explicit Xcode selection/version output. Run xcrun simctl list devices available and xcodebuild -showdestinations to choose an actual installed simulator. Do not hardcode an unverified device/runtime.
- Build using xcodebuild -project iOS/SudokuApp.xcodeproj -scheme SudokuApp -configuration Debug -sdk iphonesimulator -destination "generic/platform=iOS Simulator" CODE_SIGNING_ALLOWED=NO build. Adjust project path only if the committed project layout differs.
- Upload build/test logs and screenshots as artifacts on failure; never upload signing secrets. Distinguish “package CI green” from “native iOS app CI green”.

9. Phase 5 — Basic playable SwiftUI Sudoku (MVP)

Implementation contract

Deliverable: Interactive 9×9 Sudoku board, number keypad, and completion feedback.

Primary files/modules: iOS/SudokuApp/Views/GameView.swift, BoardView.swift, CellView.swift, NumberPadView.swift; GameViewModel.swift.

Required verification: View-model unit tests; simulator build; manual screenshot inspection from CI; UI smoke test for board and keypad.

Acceptance gate: User can launch, select editable cells, enter/clear numbers, see invalid move feedback, and complete the bundled puzzle.

- Render 9×9 grid with thicker 3×3 separators, clear visual selection, fixed/editable differentiation, and readable numbers.
- Number pad 1...9 and Clear control; tapping a fixed cell must not change it. Use explicit accessibilityIdentifier values: sudoku.board, sudoku.cell.r.c, sudoku.keypad.n, sudoku.clear, sudoku.newGame, sudoku.completed.
- Use a view model to adapt GameSession state to SwiftUI; do not duplicate Sudoku rules in views.

- Show nonblocking feedback on illegal moves and a distinct success state when the puzzle is solved. Support portrait layout and common iPhone screen sizes.
- No remote API, persistence, notes, hints, or complex animations in MVP.

10. Phase 6 — Native UI automation framework (XCUITest)

Implementation contract

Deliverable: Maintainable XCUITest suite covering the core player flows.

Primary files/modules: iOS/SudokuAppUITests/ plus page objects/screens, fixtures, helper methods; iOS CI workflow.

Required verification: xcodebuild test on an actually available iOS Simulator; capture xcresult bundle, logs, and failure screenshots.

Acceptance gate: Stable automated tests for launch, fixed cells, valid entry, clear, invalid entry, reset, and solved state.

- Use screen/page objects with accessibility identifiers, not screen coordinates or brittle text-only locators.
- Inject a deterministic puzzle through a test-only launch argument/environment configuration; production behavior must remain unchanged.
- Tests must be independent and reset game state between cases; use XCTest waits/predicates rather than arbitrary sleeps.
- Add failure attachments and upload .xcresult and readable summary to GitHub Actions.
- Record simulator runtime, Xcode version, run duration, and any flaky failures; do not automatically retry to conceal failures.

11. Phase 7 — Game quality and additional content

Implementation contract

Deliverable: Dynamic generation of playable Sudoku puzzles with unique solutions, difficulty selection, and verifiable puzzle quality.

Primary files/modules: SudokuCore solution generator, puzzle generator, solution counter/solver, difficulty model; SwiftUI game start/difficulty selection; unit tests.

Required verification: Every generated puzzle has a valid completed solution, givens consistent with that solution, and exactly one solution; seeded reproducibility tests, repeated-generation/property tests, difficulty classification tests, bounded runtime checks, and iOS build/view-model tests.

Acceptance gate: New Game generates a fresh, valid, uniquely solvable puzzle at the requested Easy/Medium/Hard difficulty; deterministic seeded generation in tests; no regressions to MVP.

- Generate a valid randomized complete 9×9 Sudoku grid, remove clues while preserving a unique solution, and use a bounded solution-counting algorithm that stops after finding two solutions.
- Define and document measurable difficulty classification before assigning Easy, Medium, or Hard labels. Use solving techniques and/or calibrated solver effort; clue count alone is not a valid difficulty guarantee. Test the classifier against known reference puzzles.

- Expose Easy/Medium/Hard selection for the next New Game; Reset must restart the current puzzle without generating a replacement. Preserve the existing starter puzzle as a deterministic test fixture, not the normal gameplay source.
- Support an injectable random seed/source so identical seeds and difficulty produce identical puzzles in unit tests; normal gameplay uses varying seeds and should avoid immediately repeating the prior puzzle.
- Bound generation attempts and execution time; if a target difficulty cannot be generated within the budget, surface an explicit failure or retry affordance rather than silently mislabeling a puzzle or substituting a fixed board.
- Keep generation and solving in platform-independent SudokuCore; do not block the SwiftUI main thread during generation. Maintain responsive UI and clear loading/error states.
- Keep Phase 6 XCUITest automation deferred for user-led implementation. Do not have Codex add XCUITest in this phase; verify UI integration using available unit/view-model tests and macOS simulator build.

12. Phase 8 — Persistence and resume

Implementation contract

Deliverable: Persist current puzzle, editable values, and completion status locally.

Primary files/modules: SudokuCore serializable snapshot, iOS persistence adapter (UserDefaults or file storage), migration/version tests.

Required verification: Round-trip tests, corrupted-data recovery, cold-launch resume UI test.

Acceptance gate: An interrupted game resumes accurately without altering givens.

- Keep storage behind a protocol so core rules remain platform-independent.
- Use versioned snapshots and explicit fallback when persisted data is malformed; no account/cloud sync.
- Test app relaunch and reset/new game storage behavior.

13. Phase 9 — Notes, hints, timer, and statistics

Implementation contract

Deliverable: Incrementally add optional Sudoku features without breaking core play.

Primary files/modules: Core notes/hint/stats models; SwiftUI controls and persistence changes; test coverage.

Required verification: Unit tests for candidate notes and hint correctness; UI tests for controls; timing/stats consistency.

Acceptance gate: Each optional feature is separately feature-complete and covered by tests.

- Notes: per-cell candidates 1...9, never mutate fixed cells; specify behavior when committing a value.
- Hints: deterministic, valid next step; define whether hints affect score/stats.
- Timer: pause/background policy and persistence defined before implementation.
- Statistics: completed games and elapsed time by difficulty; keep calculations independent of UI.

14. Phase 10 — Appium exploration and CI reliability

Implementation contract

Deliverable: Independent Python/Appium proof-of-concept against the iOS app, plus reliable CI diagnostics.

Primary files/modules: tests/appium/requirements.txt, pytest fixtures, page objects, smoke tests, README; optional dedicated workflow.

Required verification: One successful Appium iOS smoke test on a compatible macOS host and iOS Simulator; clear setup documentation.

Acceptance gate: Demonstrated trade-offs against XCUITest and a reproducible test command, or an explicitly documented infrastructure limitation.

- Use Appium 2, XCUITest driver, Python pytest, and accessibility identifiers shared with XCUITest.
- Appium server and XCUITest driver require macOS/Xcode for an iOS target; do not assume Windows can host iOS automation.
- Run only when macOS runner permissions, simulator boot, WebDriverAgent signing/build, and runtime availability are confirmed.
- Collect Appium server logs, screenshots, device logs, and test reports. Prefer XCUITest as the primary CI framework if Appium is unreliable on hosted runners.

15. GitHub Actions — test matrix, cost, and reporting

Stage	Runner / trigger	Commands / artifacts
Phase 1–3 core	Windows local; macos-15 push/PR	swift build; swift test; Swift Testing output
Phase 4 iOS build	macOS push/PR, after project exists	xcodebuild -version, simctl list, xcodebuild simulator build, logs
Phase 5–9 UI tests	macOS PR/manual, available simulator	xcodebuild test, .xcresult, failure screenshots
Phase 10 Appium	macOS manual/experimental	Appium server, pytest, logs/screenshots

- Use public-repository standard GitHub-hosted runners for free Actions minutes under current GitHub policy; avoid larger paid runners and monitor usage/policy changes.
- Pin major action versions at minimum; prefer reviewed pinned SHAs for mature workflows. Use least-privilege permissions (contents: read), concurrency cancellation for superseded PR builds, and job timeouts.
- Do not assume hosted runners provide an interactive Mac desktop. Simulator tests run unattended and may require selecting a supported Xcode/runtime pair.
- Split fast core tests from expensive iOS UI tests. Core tests should run on every PR; UI tests may run on PR or workflow_dispatch after runner support is proven.
- A failed build/test blocks phase acceptance. Upload diagnostics on failure; do not mark a phase complete merely because the YAML workflow started.

16. Testing on a personal iPhone — realistic limitations

A physical iPhone can eventually provide valuable manual validation for touch input, keyboard/gesture behavior, display scaling, and real-device performance. However, GitHub-hosted macOS runners do not have USB access to your personal iPhone. An iOS Simulator build is not installable directly on a physical iPhone.

- For direct development installation, Apple's supported workflow uses Xcode on a Mac with code signing and a connected/paired device. A free Apple Account may support personal development signing through Xcode, with limitations; a paid membership is generally needed for TestFlight/App Store distribution.
- A GitHub-hosted macOS runner can build simulator binaries without signing; it cannot automatically provision your iPhone or supply an official free distribution route from Windows.
- Do not store Apple credentials, provisioning profiles, or device IDs in a public repository.
- Decision gate: revisit real-device deployment only if legitimate Mac access or another supported signing/distribution route becomes available. For the zero-cost Windows-only MVP, simulator CI is the acceptance target.

17. Phase dependencies and release gates

Phase	Depends on	Approval condition
1	None	Windows core and macOS core CI green — DONE
2	1	Board/rules tests pass
3	2	Deterministic session and puzzle tests pass
4	1–3	Native iOS project builds in macOS CI
5	3–4	Playable MVP with simulator verification
6	5	XCUITest suite stable in CI
7	5–6	Unique dynamic puzzles and difficulty validated
8	5–6	Resume/recovery tested
9	7–8	Optional features and tests accepted
10	4–6	Appium feasibility demonstrated

18. References and maintenance

- Swift Windows installation: https://www.swift.org/install/windows/
- Swift Package Manager: https://www.swift.org/documentation/package-manager/
- GitHub Actions macOS runners: https://docs.github.com/en/actions/reference/runners/github-hosted-runners
- Xcodebuild: https://developer.apple.com/documentation/xcode/building-your-app-from-the-command-line

- Appium XCUITest driver: https://appium.github.io/appium-xcuitest-driver/
- Update this plan when versions, CI images, or phase decisions change. Keep acceptance criteria explicit; never silently reinterpret a phase.